

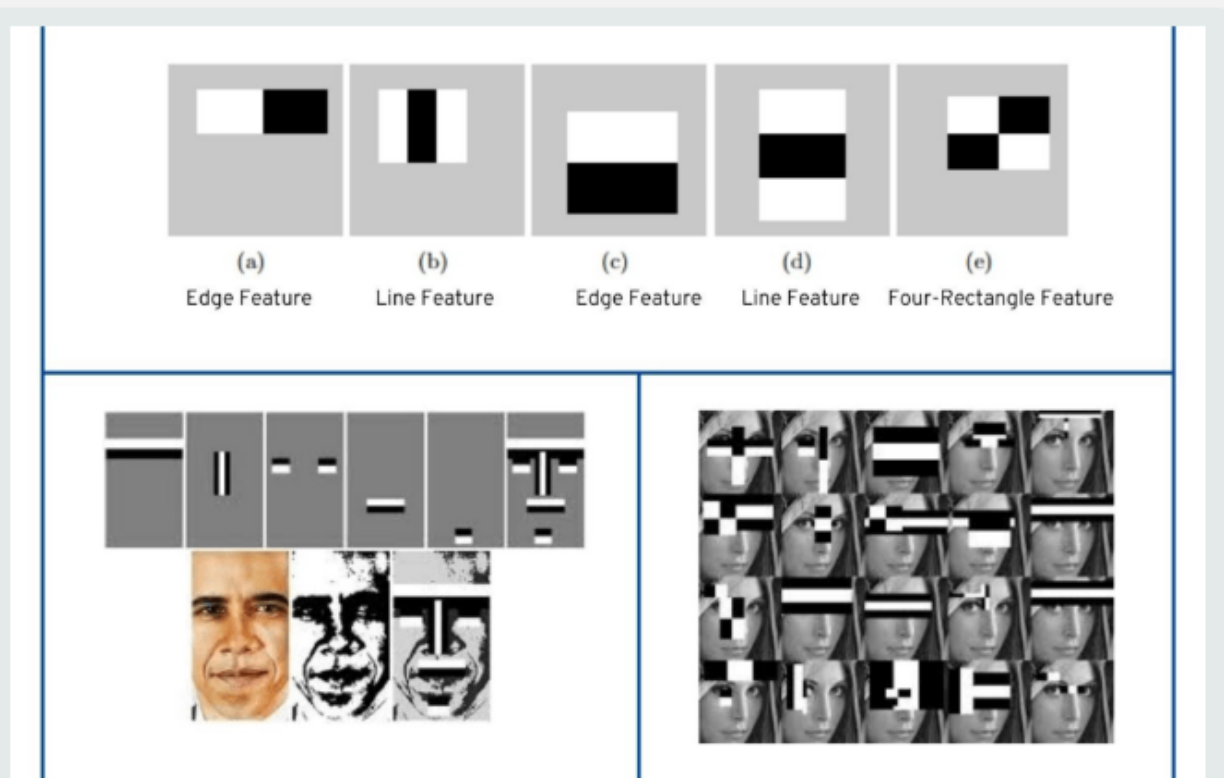
Лабораторная работа №7

Распознавание лиц в видеопотоке

Как работает распознавание лиц: метод Виолы-Джонса

Один из способов распознать образ — найти контур объекта и исследовать его свойства. По этому принципу работает метод Виолы-Джонса с использованием признаков Хаара, который придумал венгерский математик Альфред Хаар.

Признаки — это набор геометрических фигур с черно-белым узором, их еще называют маски. Они помогают найти границы какой-либо формы, например очертания лица, линии бровей, носа или рта.



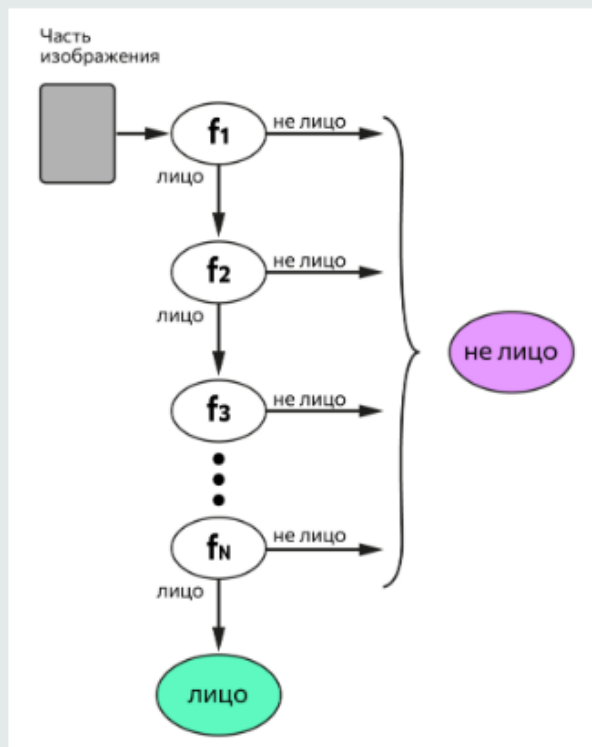
Маски (признаки Хаара) накладываются на фотографии и «находят» на них лица.

В алгоритме Виолы-Джонса маски накладываются на разные части кадра, а программа определяет, может ли в них находиться объект. Работает это так:

1. Классификатор (алгоритм, который будет искать объект в кадре) обучают на фотографиях лиц и получают пороговое значение — его превышение будет сигнализировать о том, что в кадре есть лицо.
2. В классификатор загружают изображение, на котором будут искать лицо.
3. Классификатор накладывает на него маски и отдельно складывает яркость пикселей, попавших в белую часть маски, и яркость пикселей, попавших в черную часть маски. Потом из первого значения он вычитает второе.

Результат сравнивается с пороговой величиной.

Если результат меньше пороговой величины, значит, в части кадра нет лица, и алгоритм заканчивает свою работу. А если больше, он переходит к следующей части кадра.



Так происходит классификация: подобласть изображения, которая успешно проходит через весь каскад классификаторов $f_1, \dots, f_n$, классифицируется как лицо

Алгоритмы OpenCV

У изображения лица есть свои характеристики:

1. Темные и светлые участки и зоны (темные — глаза, губы; светлые — лоб, щеки, подбородок).
2. Лица всех людей устроены по одному принципу (глаза — на одной линии, под глазами — нос, под носом — губы, под губами — подбородок).

Это значит, что можно подобрать такой набор масок и составить такой классификатор, который будет учитывать эти особенности.

Для этого можно использовать OpenCV — библиотеку алгоритмов компьютерного зрения и обработки изображений. Реализована она на C/C++, также разрабатывается для Python, Java, Ruby, Matlab, Lua и других языков.

Модернизируем код для распознавания лиц так, чтобы в реальном времени обнаруживать лица в кадре, например, на видео.

Шаг 1. Создайте объект для захвата видеострима:

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
capture_io = cv2.VideoCapture('Download.mp4')
```

Шаг 2. Считывать кадры с видеоустройства будем в вечном цикле: `cv2.VideoCapture.read()` возвращает булево значение об успешном считывании из потока (оно нам не нужно) в паре с самой картинкой. Преобразуем ее в 8-битную матрицу так же, как и раньше:

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
capture_io = cv2.VideoCapture('Download.mp4')
while True:
    _, img = capture_io.read()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Шаг 3. Остается переписать только конец цикла — чтобы у него было условие выхода, например, по нажатию на «q» (от quit, «выйти»). `cv2.waitKey(time)` 10 мс ожидает ввода с клавиатуры юникод-символа, который приведет к закрытию стрима и программы.

```
import cv2
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
capture_io = cv2.VideoCapture('Download.mp4')
while True:
    _, img = capture_io.read()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
        cv2.imshow('img', img)
    if cv2.waitKey(10) & 0xFF == ord('q'):
        break
capture_io.release()
cv2.destroyAllWindows()
```

Шаг 4. Код готов, запускаем, и наблюдаем результат вживую:

```
import cv2

face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')

capture_io = cv2.VideoCapture('Download.mp4')

while True:
    _, img = capture_io.read()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
        cv2.imshow('img', img)
        if cv2.waitKey(10) & 0xFF == ord('q'):
            break
    capture_io.release()
cv2.destroyAllWindows()
```

